

AI Assistant Coding

Assignment-15

Name : Abhishek Yadav

Hall Ticket No. : 2303A510C3

Batch No. : 06

Task 1– Real-Time Application: Online Food Ordering API

Scenario:

Design a simple API for an online food ordering system.

Requirements:

- Endpoints required:
- GET /menu → List available dishes
- POST /order → Place a new order
- GET /order/{id} → Track order status
- PUT /order/{id} → Update an existing order (e.g., change items)
- DELETE /order/{id} → Cancel an order
- **AI should generate:**
- REST API code
- Suggested improvements (like authentication, pagination) **Expected Output:**
- Fully working API simulating a food ordering backend.

Prompt:

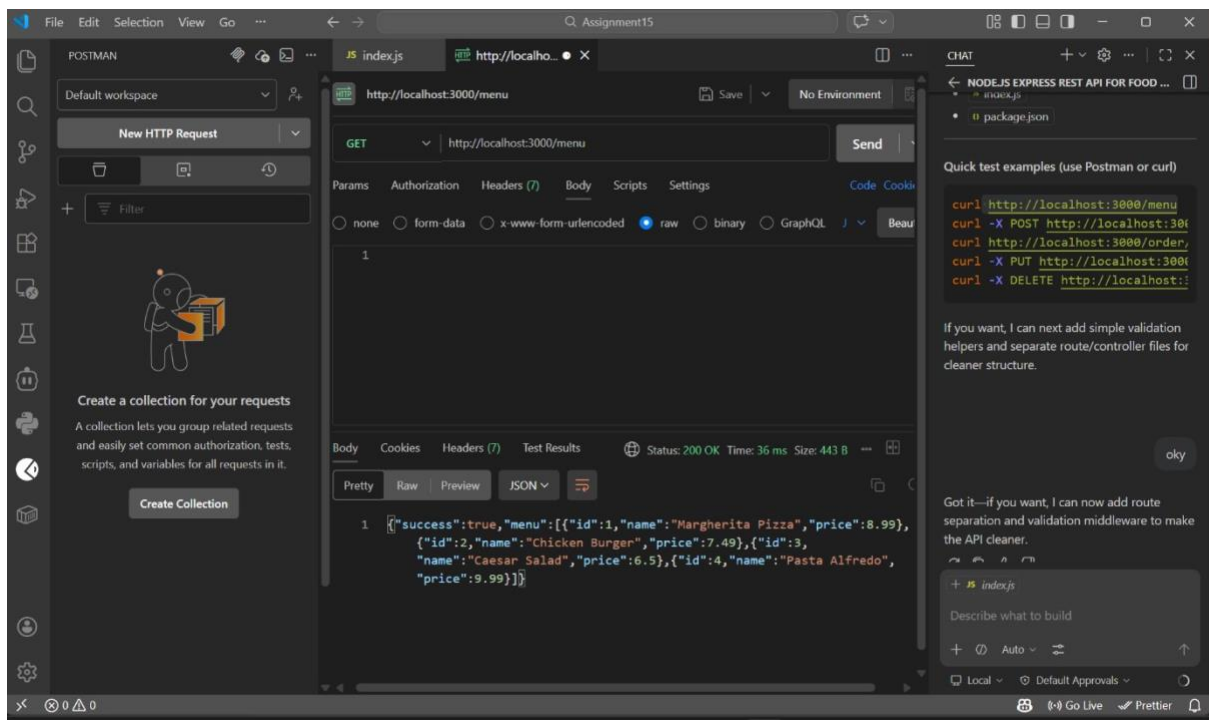
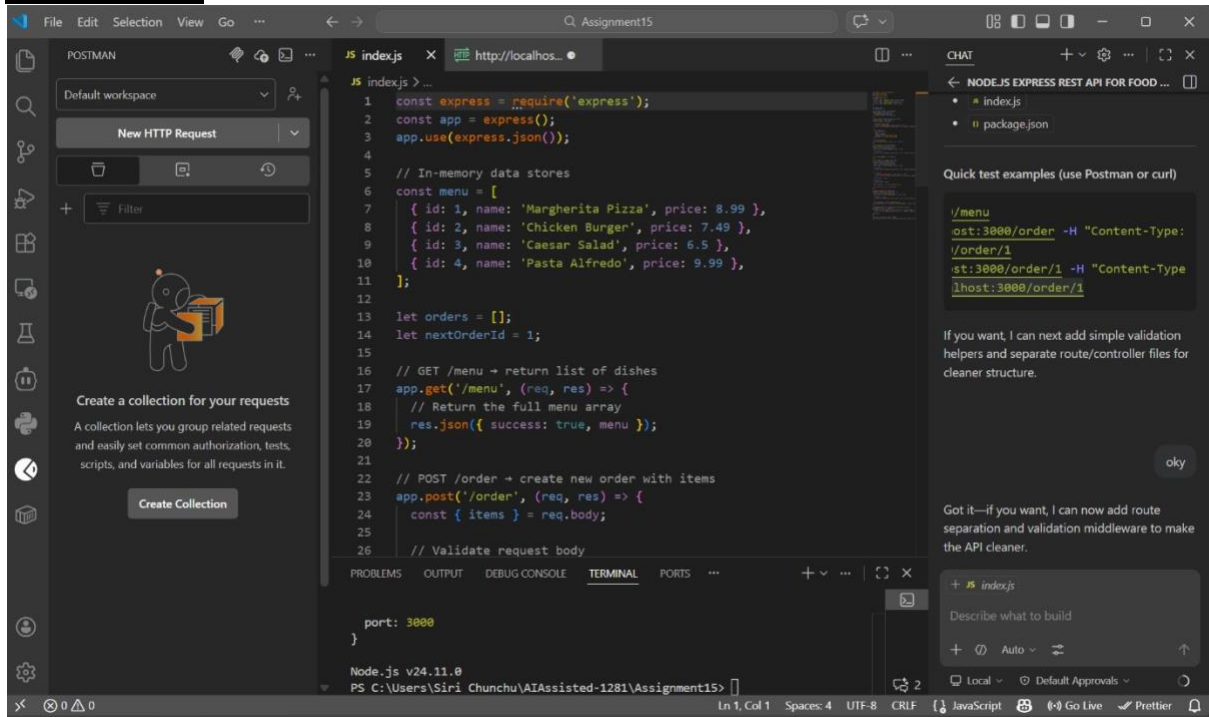
Create a Node.js Express REST API for an online food ordering system.

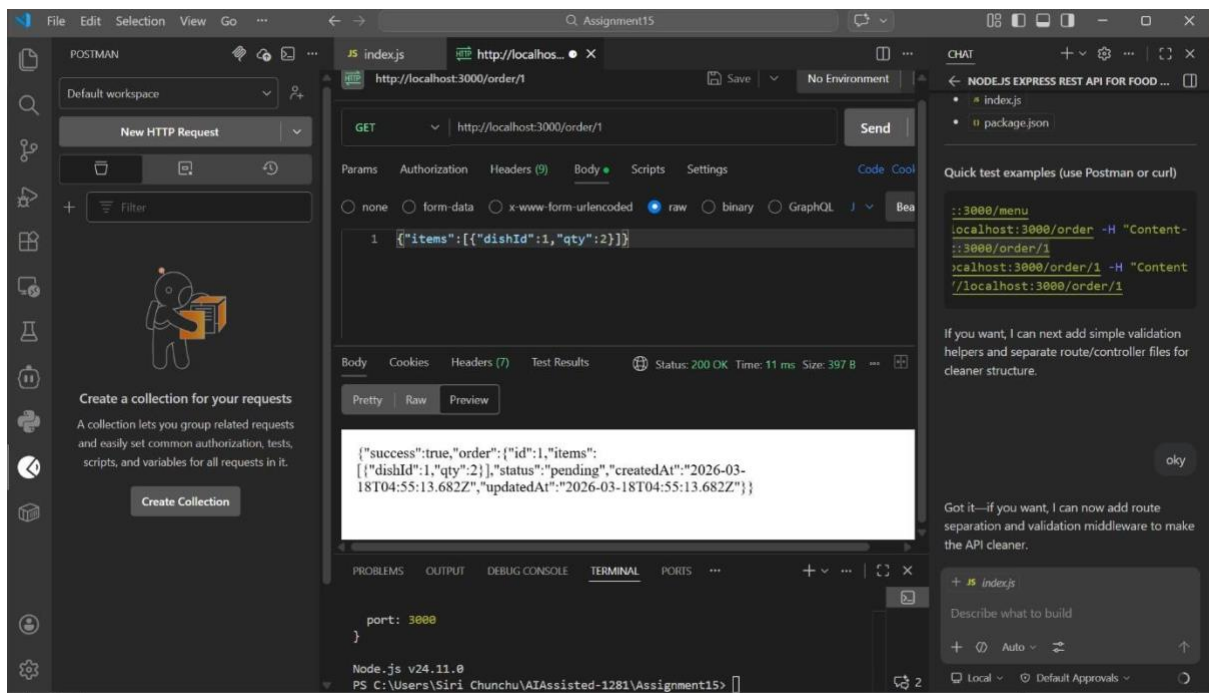
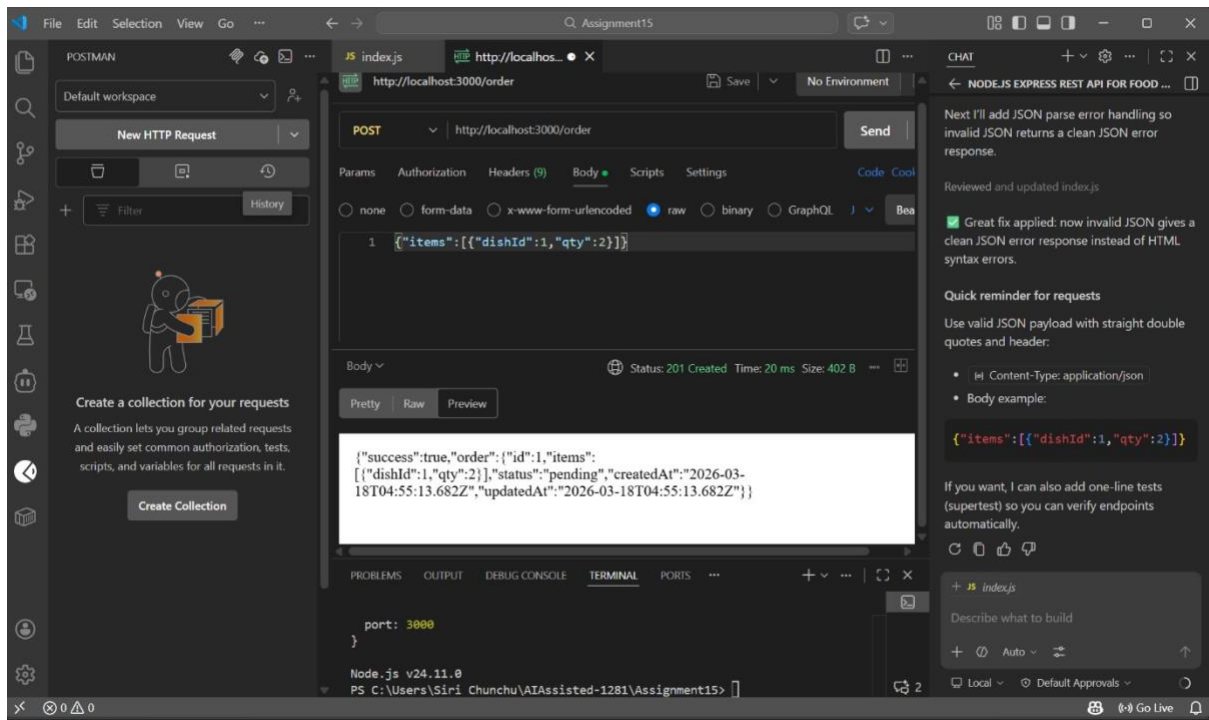
Requirements:

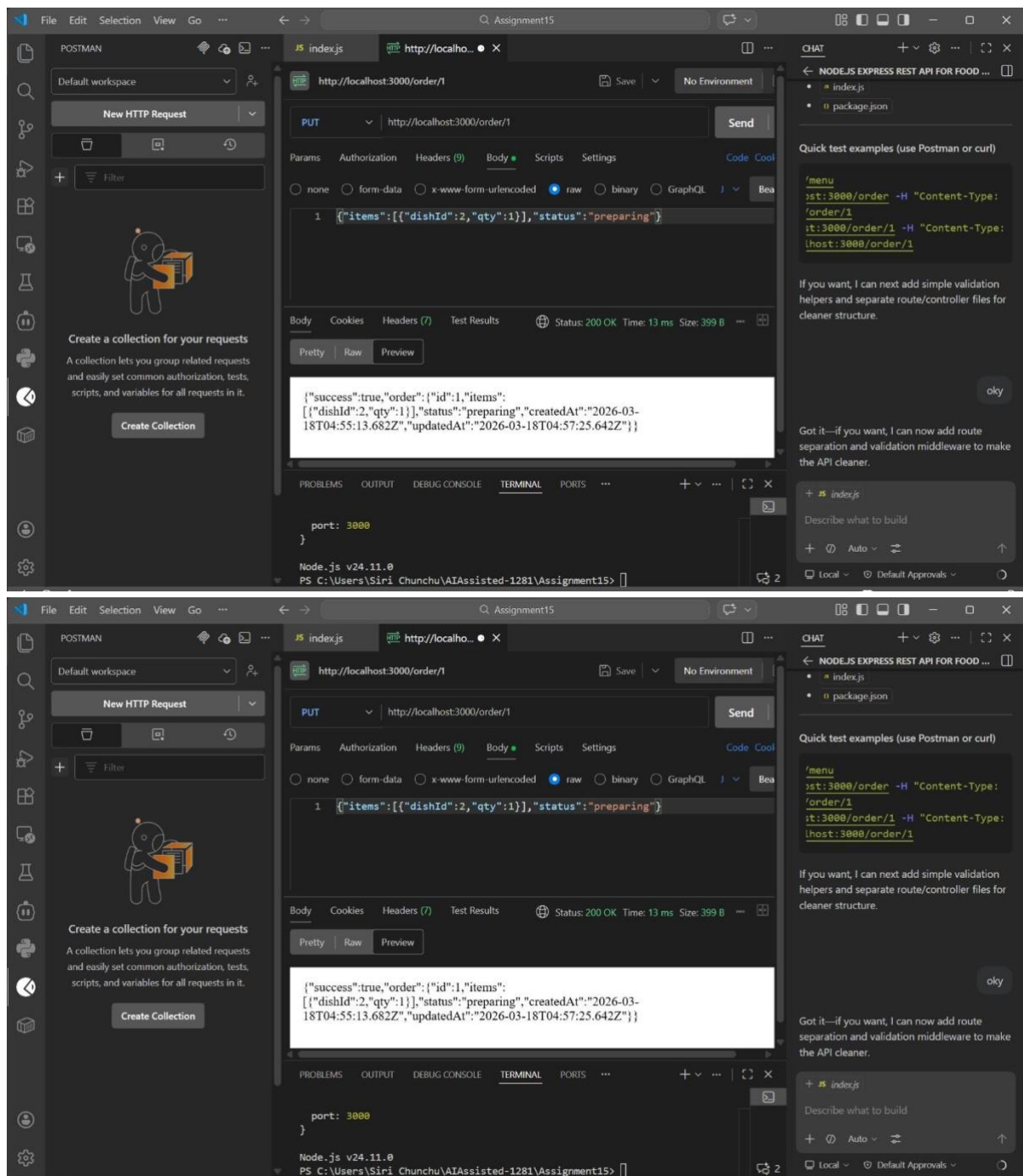
- GET /menu → return list of dishes
- POST /order → create new order with items
- GET /order/:id → get order status
- PUT /order/:id → update order items
- DELETE /order/:id → cancel order

Use in-memory arrays. Add proper comments for each endpoint.

AI Generated:







Explanation

This API simulates a basic online food ordering system using REST principles.

The **GET /menu** endpoint retrieves available dishes.

The **POST /order** endpoint allows users to place a new order by sending selected items.

The **GET /order/:id** endpoint helps in tracking the status of an order.

The **PUT /order/:id** endpoint is used to update order details such as items.

The **DELETE /order/:id** endpoint cancels an existing order.

All data is stored in an in-memory array, making it simple and fast for testing.

Task 2 – Employee Payroll API

Task:

Create an API for managing employees and their salaries.

Instructions:

• Endpoints required:

- GET /employees → List all employees
- POST /employees → Add a new employee with salary details
- PUT /employees/{id}/salary → Update salary of an employee • DELETE /employees/{id} → Remove employee from system

• Use AI to:

- Suggest data model structure.
- Add comments/docstrings for all endpoints.

Expected Output:

- Payroll management API with validation
- Secure and structured data handling
- Clear API documentation

Prompt:

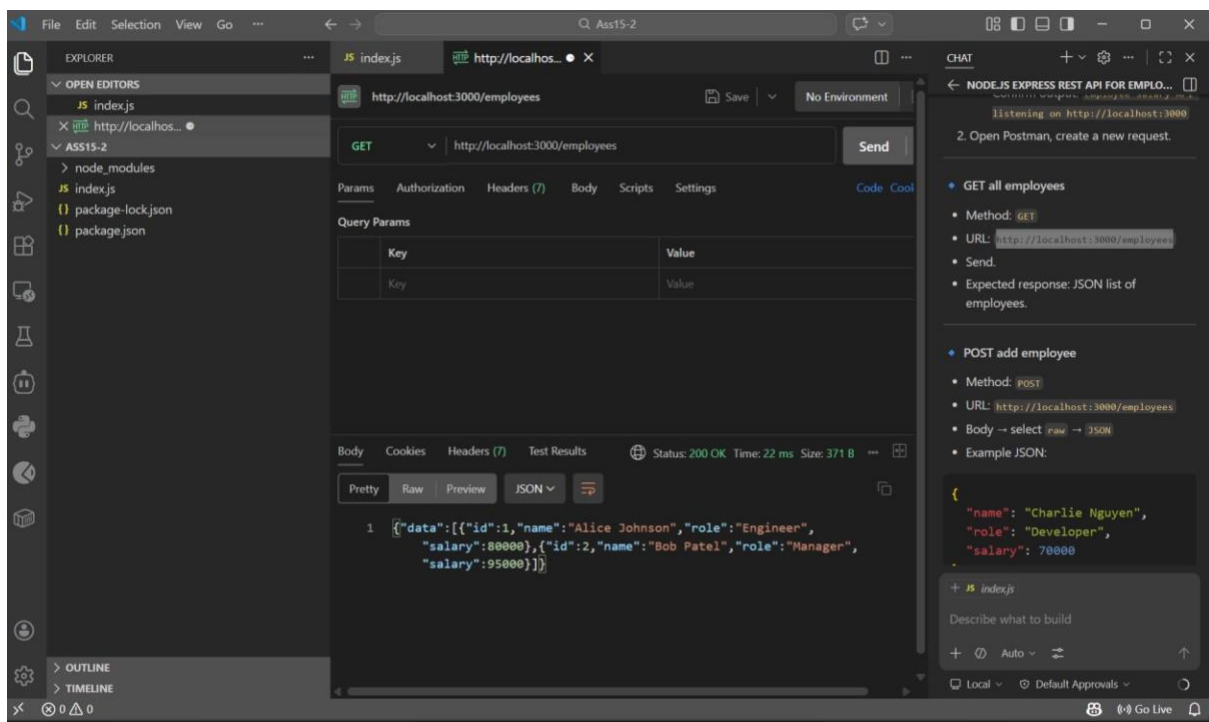
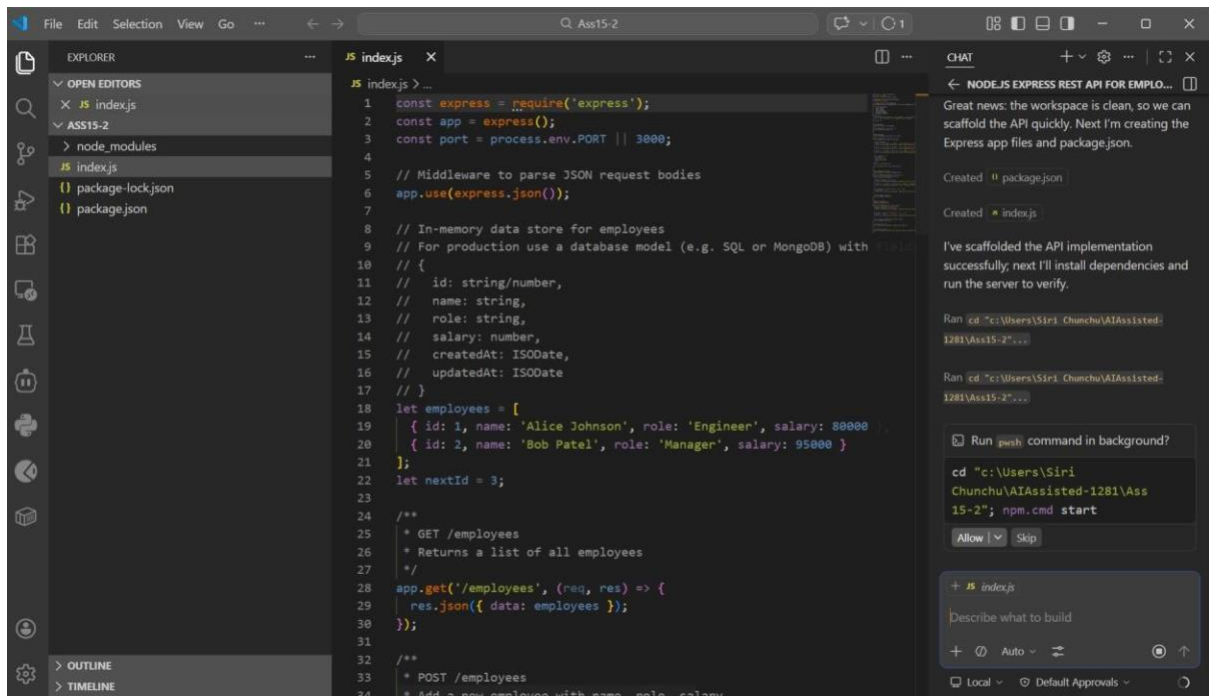
Create a Node.js Express REST API to manage employees and salaries.

Requirements:

- GET /employees → list all employees
- POST /employees → add employee with name, role, salary
- PUT /employees/:id/salary → update salary
- DELETE /employees/:id → remove employee Add validation and comments for each endpoint.

Also suggest a proper data model.

AI Generated:



Visual Studio Code interface showing a REST client request to `http://localhost:3000/employees` using the `POST` method. The request body is a JSON object:

```
{  "name": "Charlie Nguyen",  "role": "Developer",  "salary": 70000}
```

The response status is `201 Created`. The response body is a JSON array:

```
[{"data": [{"id": 1, "name": "Alice Johnson", "role": "Engineer", "salary": 80000}, {"id": 2, "name": "Bob Patel", "role": "Manager", "salary": 95000}]}]
```

The right sidebar shows the `CHAT` panel with the following content:

- GET all employees**
 - Method: `GET`
 - URL: `http://localhost:3000/employees`
 - Send.
 - Expected response: JSON list of employees.
- POST add employee**
 - Method: `POST`
 - URL: `http://localhost:3000/employees`
 - Body → select `raw` → `JSON`
 - Example JSON:

```
{  "name": "Charlie Nguyen",  "role": "Developer",  "salary": 70000}
```
 - Send → should return `201` with new

Below the chat panel, there is a section for `+ JS index.js` with the text "Describe what to build" and a "Go Live" button.

Visual Studio Code interface showing a REST client request to `http://localhost:3000/employees/1/salary` using the `PUT` method. The request body is a JSON object:

```
{  "salary": 85000}
```

The response status is `200 OK`. The response body is a JSON array:

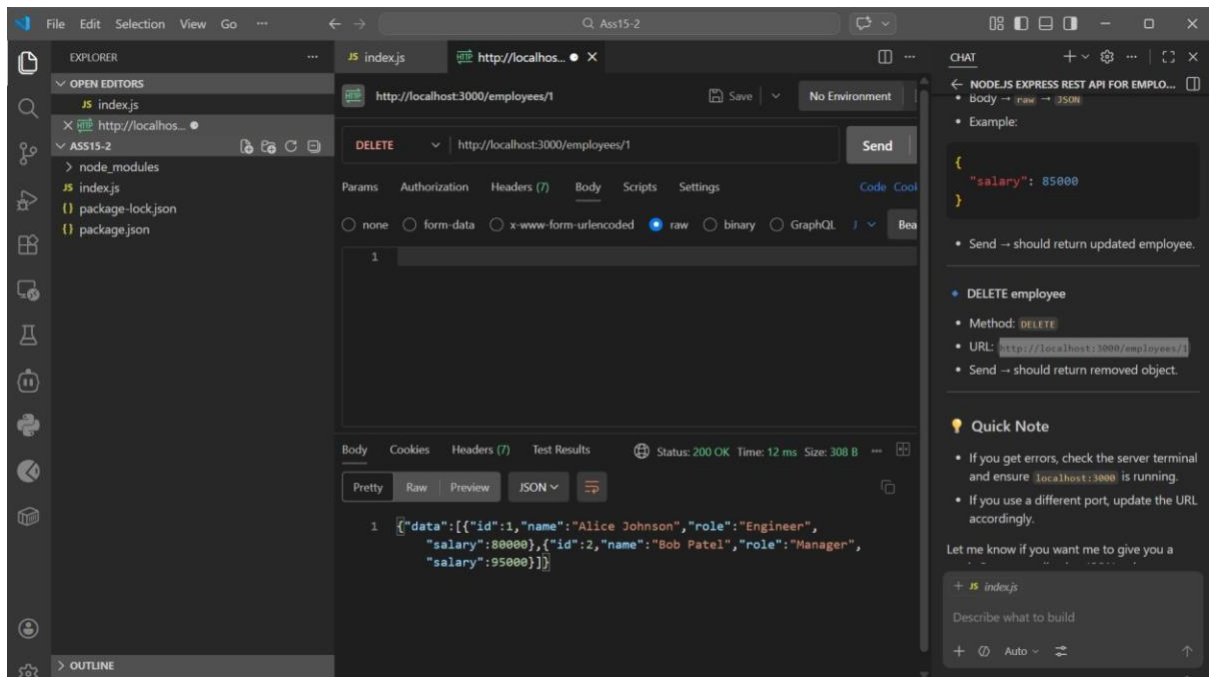
```
[{"data": [{"id": 1, "name": "Alice Johnson", "role": "Engineer", "salary": 80000}, {"id": 2, "name": "Bob Patel", "role": "Manager", "salary": 95000}]}]
```

The right sidebar shows the `CHAT` panel with the following content:

- PUT update salary**
 - Method: `PUT`
 - URL: `http://localhost:3000/employees/1/salary`
 - Body → `raw` → `JSON`
 - Example:

```
{  "salary": 85000}
```
 - Send → should return updated employee.

Below the chat panel, there is a section for `+ JS index.js` with the text "Describe what to build" and a "Go Live" button.



Explanation

This API manages employee records and salary details.

The **GET /employees** endpoint retrieves all employee data.

The **POST /employees** endpoint adds a new employee with salary information.

The **PUT /employees/:id/salary** endpoint updates the salary of a specific employee.

The **DELETE /employees/:id** endpoint removes an employee from the system.

Basic validation ensures correct data input before processing.

Task 3: Student Information REST API

Scenario:

A university wants to digitize student records for easy retrieval. You need to build a Student Management API that allows:

- Adding a new student (name, roll number, course, year)
- Fetching details of all students
- Updating student information
- Deleting a student record

Implementation Instructions:

1. Use Node.js + Express (or Python Flask/Django REST) with GitHub Copilot to auto-generate routes.
2. Store student data in an in-memory array or SQLite/PostgreSQL

database.

Test API endpoints with Postman.

Prompt:

Create a Student Management REST API using Node.js and Express.

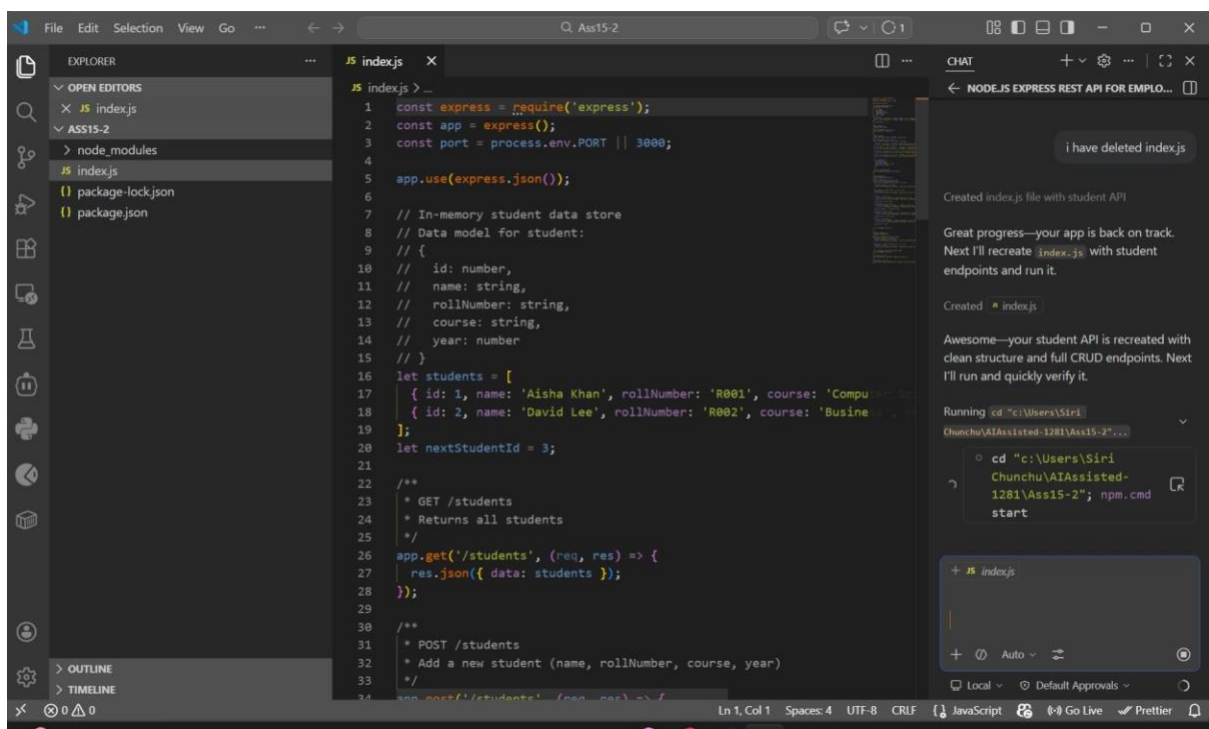
Features:

- Add student (name, roll number, course, year)
- Get all students
- Update student details
- Delete student

Store data in an in-memory array.

Add comments and clean structure.

AI Generated:



```
1 const express = require('express');
2 const app = express();
3 const port = process.env.PORT || 3000;
4
5 app.use(express.json());
6
7 // In-memory student data store
8 // Data model for student:
9 // {
10 //   id: number,
11 //   name: string,
12 //   rollNumber: string,
13 //   course: string,
14 //   year: number
15 // }
16 let students = [
17   { id: 1, name: 'Aisha Khan', rollNumber: 'R001', course: 'Computer Science', year: 2020 },
18   { id: 2, name: 'David Lee', rollNumber: 'R002', course: 'Business Administration', year: 2021 }
19 ];
20 let nextStudentId = 3;
21
22 /**
23  * GET /students
24  * Returns all students
25  */
26 app.get('/students', (req, res) => {
27   res.json({ data: students });
28 });
29
30 /**
31  * POST /students
32  * Add a new student (name, rollNumber, course, year)
33  */
34 app.post('/students', (req, res) => {
```

CHAT

← NODE.JS EXPRESS REST API FOR EMPLO...

i have deleted index.js

Created index.js file with student API

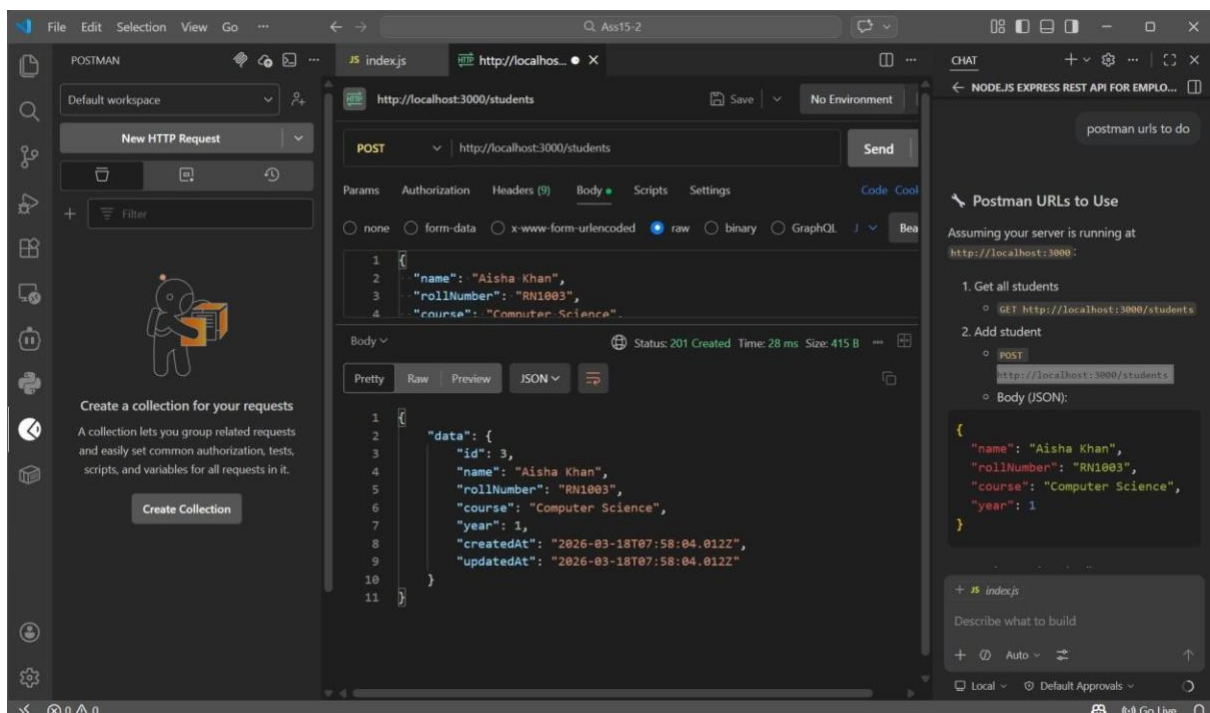
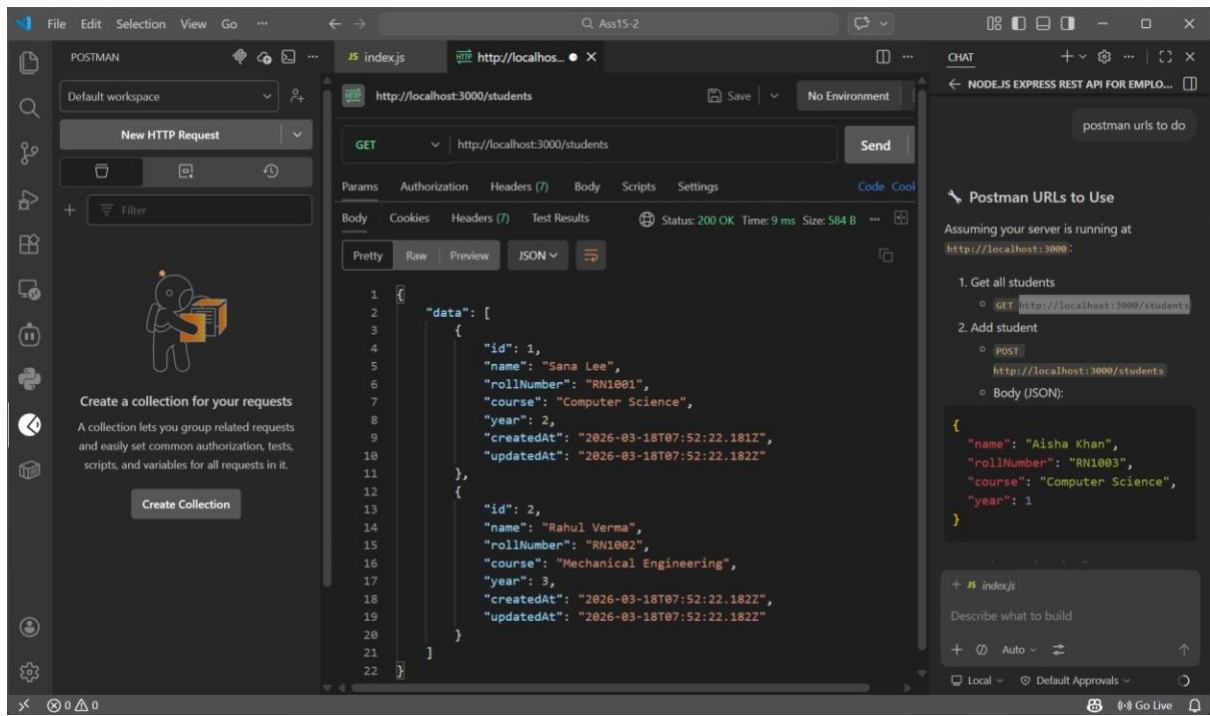
Great progress—your app is back on track. Next I'll recreate index.js with student endpoints and run it.

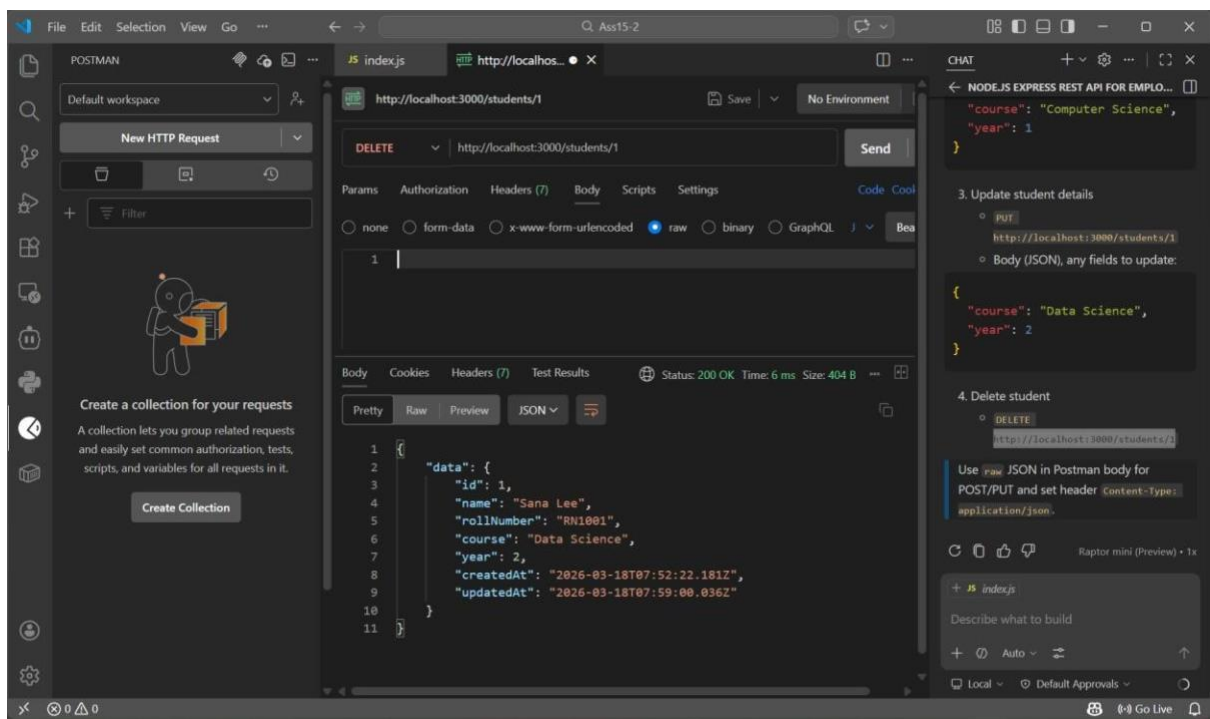
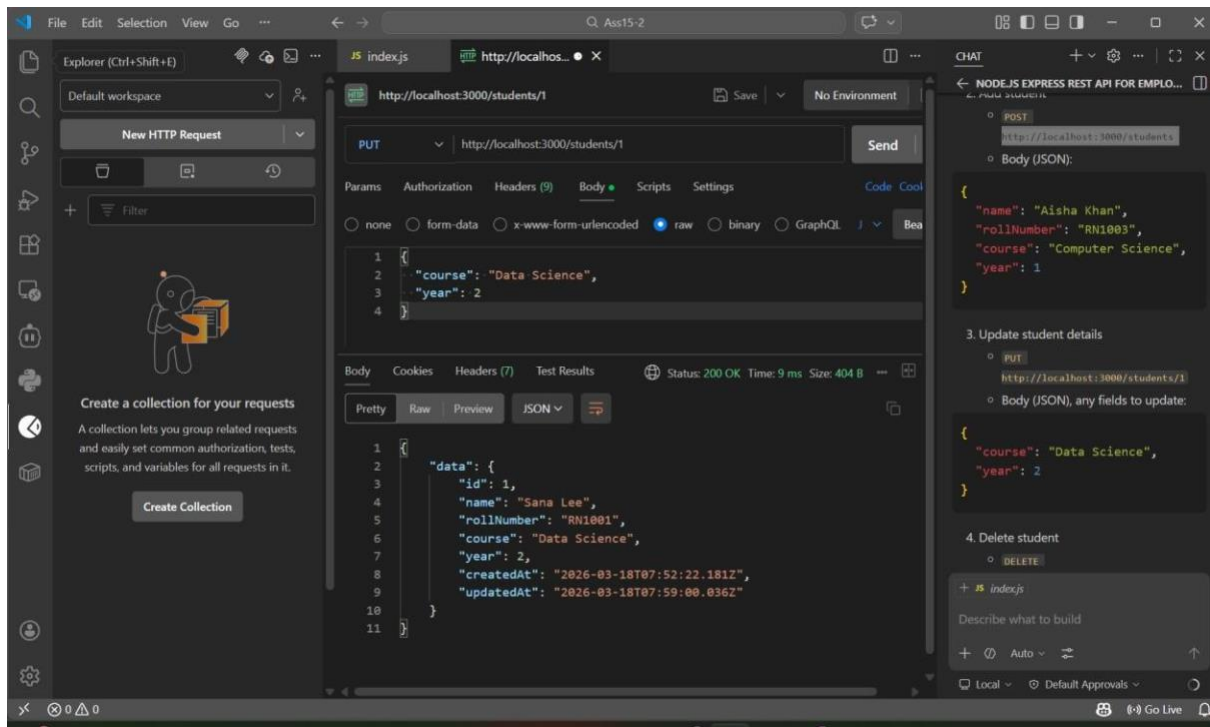
Created index.js

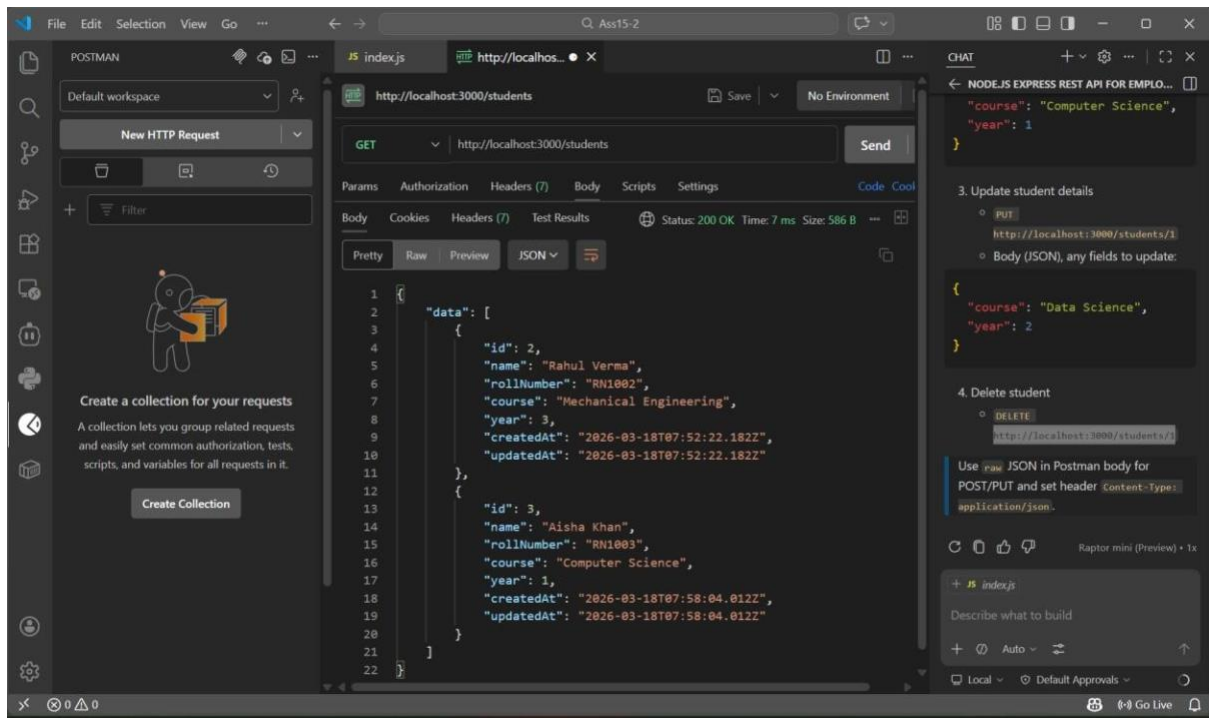
Awesome—your student API is recreated with clean structure and full CRUD endpoints. Next I'll run and quickly verify it.

Running cd "c:\Users\Siri\Chunchu\AIAssisted-1281\Ass15-2"; npm.cmd start

+ JS index.js







Explanation

This API is designed to manage student records in a university system.

The **POST** `/students` endpoint adds new student details.

The **GET** `/students` endpoint retrieves all student records.

The **PUT** `/students/:id` endpoint updates existing student information.

The **DELETE** `/students/:id` endpoint deletes a student record.

Data is stored in an in-memory array for simplicity.

Task 4: Weather Information API Integration

Scenario:

Students often want to check weather before attending campus activities. Build a backend service that fetches real-time weather data from a public API (e.g., OpenWeatherMap).

Implementation Instructions:

1. Create endpoints like:

- `/weather/current/:city` → fetches current weather
- `/weather/forecast/:city` → fetches 5-day forecast

2. Use GitHub Copilot to generate code that calls the external API.

Return results in structured JSON format with temperature, humidity, and condition.

Prompt:

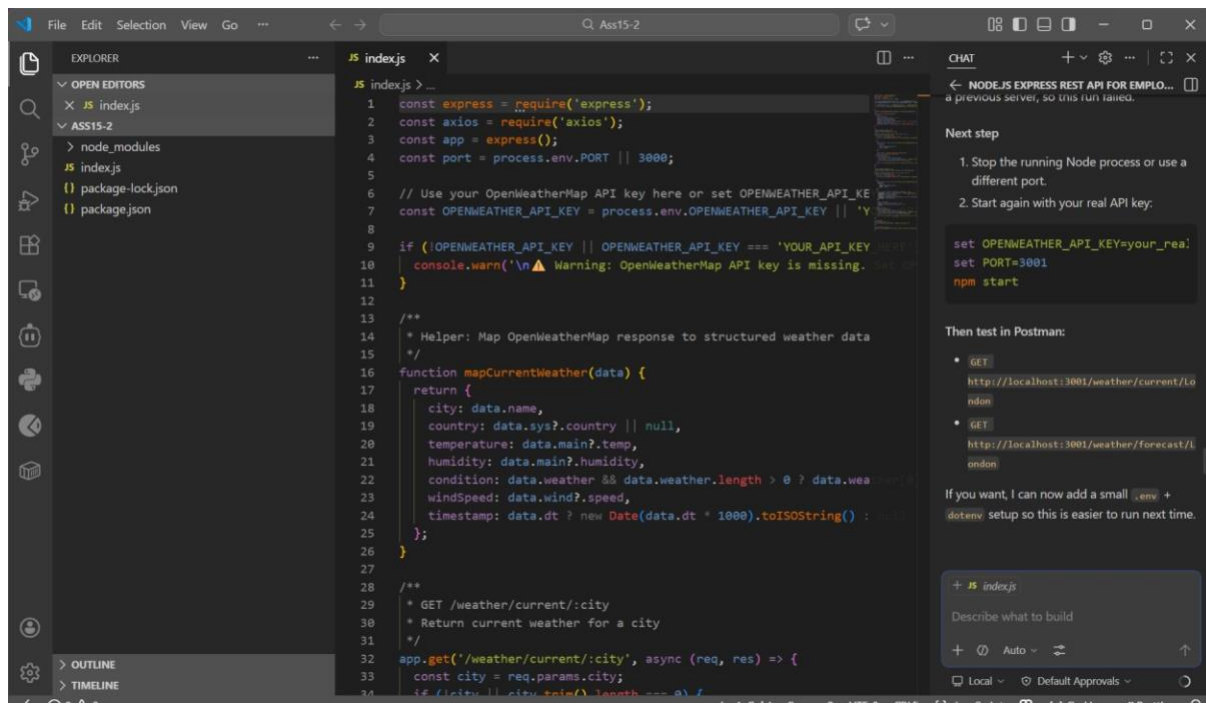
Create a Node.js Express API that integrates with OpenWeatherMap API.

Endpoints:

- /weather/current/:city → return temperature, humidity, condition
- /weather/forecast/:city → return 5-day forecast Use axios for API calls and return structured JSON.

Include error handling and comments.

AI Generated:



```
1 const express = require('express');
2 const axios = require('axios');
3 const app = express();
4 const port = process.env.PORT || 3000;
5
6 // Use your OpenWeatherMap API key here or set OPENWEATHER_API_KEY
7 const OPENWEATHER_API_KEY = process.env.OPENWEATHER_API_KEY || 'YOUR_API_KEY_HERE';
8
9 if (!OPENWEATHER_API_KEY || OPENWEATHER_API_KEY === 'YOUR_API_KEY_HERE') {
10   console.warn('\n⚠ Warning: OpenWeatherMap API key is missing. Set it in your .env file or as an environment variable.');
11 }
12
13 /**
14  * Helper: Map OpenWeatherMap response to structured weather data
15  */
16 function mapCurrentWeather(data) {
17   return {
18     city: data.name,
19     country: data.sys?.country || null,
20     temperature: data.main?.temp,
21     humidity: data.main?.humidity,
22     condition: data.weather[0] ? data.weather[0].description : null,
23     windSpeed: data.wind?.speed,
24     timestamp: data.dt ? new Date(data.dt * 1000).toISOString() : null;
25   };
26 }
27
28 /**
29  * GET /weather/current/:city
30  * Return current weather for a city
31  */
32 app.get('/weather/current/:city', async (req, res) => {
33   const city = req.params.city;
34   if (!city || city.trim().length === 0) {
```

Next step

1. Stop the running Node process or use a different port.
2. Start again with your real API key:

```
set OPENWEATHER_API_KEY=your_real_key
set PORT=3001
npm start
```

Then test in Postman:

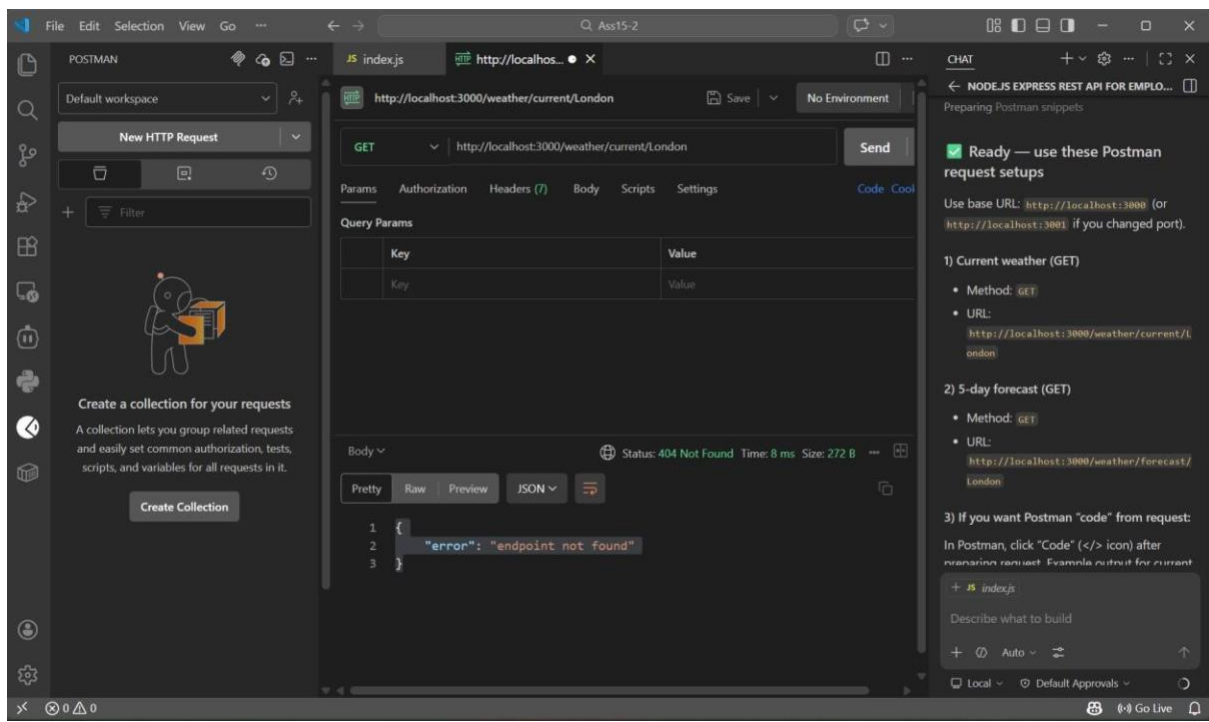
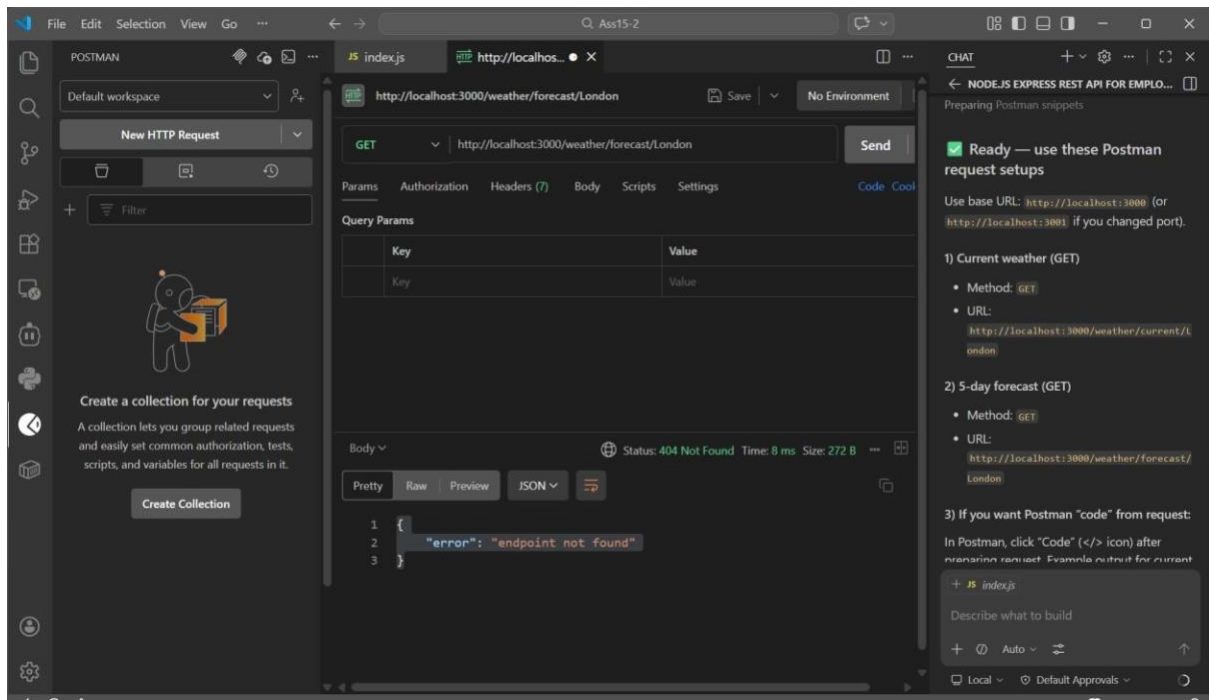
- GET `http://localhost:3001/weather/current/London`
- GET `http://localhost:3001/weather/forecast/London`

If you want, I can now add a small `.env` + `dotenv` setup so this is easier to run next time.

+ JS index.js

Describe what to build

+ 🔍 Auto - ⚙️



Explanation

This API integrates with an external service (OpenWeatherMap) to fetch real-time weather data.

The `/weather/current/:city` endpoint retrieves current weather details such as temperature, humidity, and condition.

The `/weather/forecast/:city` endpoint provides forecast data.

Axios is used to make HTTP requests to the external API, and structured JSON responses are returned to the user.

Task 5: Delete Item

Instructions:

- Use AI to create a DELETE endpoint to remove an item by index or ID.

Starter Code:

```
# DELETE /items/<int:index>

@app.route('/items/<int:index>', methods=['DELETE']) def
delete_item(index): if index < 0 or index >= len(items): return
jsonify({"error": "Item not found"}), 404 removed_item =
items.pop(index) return jsonify({"message": "Item deleted",
"item": removed_item})
```

Expected Output:

```
• DELETE /items/0 →
{"message": "Item deleted", "item": {"name": "Notebook", "price": 250}}
GET /items → []
```

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

Prompt:

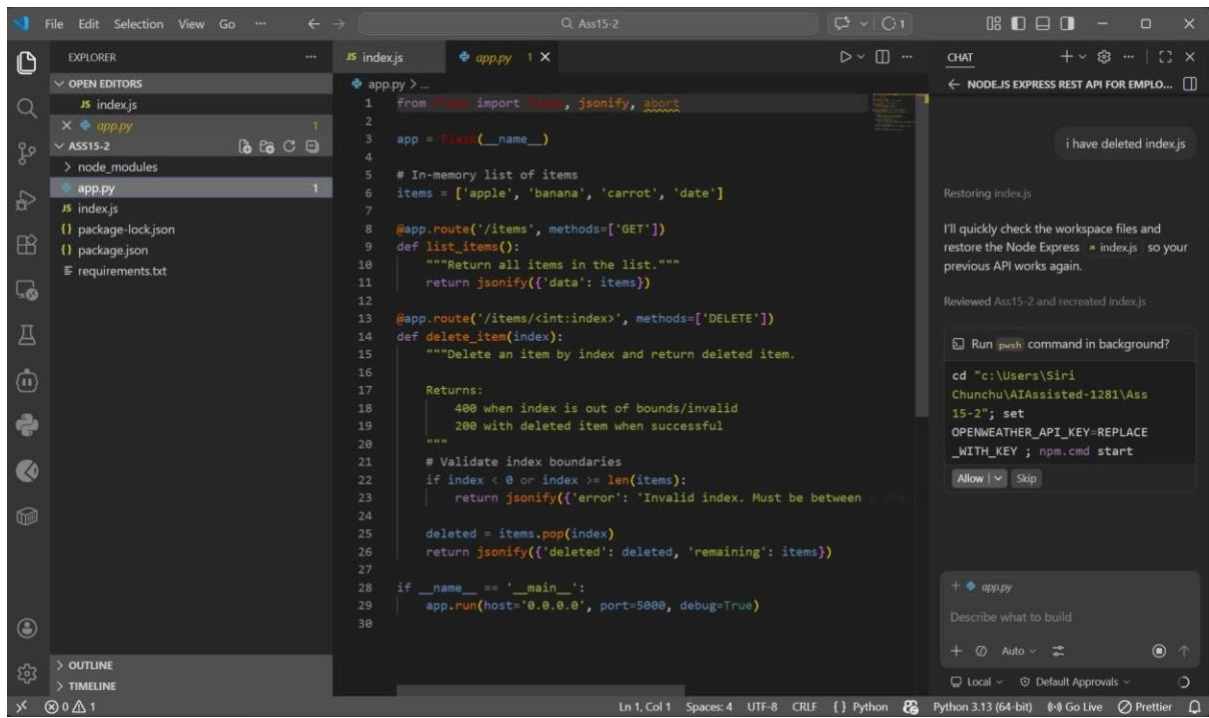
Create a Flask API endpoint to delete an item from a list by index.

Requirements:

- DELETE /items/<int:index>
- Return error if index invalid - Return deleted item in response

Add proper comments.

AI Generated:



DELETE

⌵

http://127.0.0.1:5000/items/0

Params

Authorization

Headers (2)

Body

Pre-request Script

Tests

☐ none

☐ form-data

☒ x-www-form-urlencoded

☐ 

☐ binary

☐ GraphQL

1

{

2

"message": "Item deleted",

3

"item": {

4

"name": "Notebook",

5

"price": 250

6

}

7

}

Body

Cookies

Preseies

Test (1)

Test Results

Pretty

Raw

Preview

Visualize

JSON ⌵

⋮

1

{

2

}

GET

⌵

http://127.0.0.1:5000/items

Params

Authorization

Headers (2)

Body

Pre-request Script

Tests

☐ none

☐ form-data

☒ x-www-form-urlencoded

☐

☐ binary

☐ GraphQL

1

}

Body

Cookies

Preseies

Test (1)

Test Results

Pretty

Raw

Preview

Visualize

JSON ⌵

⋮

Explanation

This API demonstrates how to delete an item from a list using a DELETE request.

The **DELETE** `/items/<index>` endpoint removes an item based on its index.

It includes error handling to check if the index is valid.

If valid, the item is removed and returned in the response.